FAQ

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Pages . . . 1
When in doubt . . . 1
Missing input references (diagnostics.out) . . . 2
Errors written to stdout . . . 3
Failure modes that are not text errors . . . 4
Related . . . 4
What to compare . . . 4
Symptom: precip is wrong . . . 5
Symptom: surq_gen is too high or too low . . . 5
Symptom: ET is too high or too low . . . 5
Symptom: management never fires . . . 5
Symptom: percolation is zero or saturated . . . 6
Symptom: bad latq, qtile, or surq_runon . . . 6
Symptom: ru_aa.txt and basin_wb_aa.txt disagree . . . 6
Workflow . . . 6
Related . . . 6
Build with `-j` . . . 6
Choose an optimized build type . . . 7
Daily versus sub-daily routing . . . 7
The cost of enabling many outputs . . . 7
Object count . . . 7
I/O . . . 8
Profiling . . . 8
Summary . . . 8

When a SWAT+ run does not produce what you expect, the first three files to read are `simulation.out`, `diagnostics.out`, and `checker.out`. Their format is described in Checker and diagnostics.

Pages

- Common runtime errors. The error strings the model writes to its diagnostic files, what they mean, and what to change.
- Water balance and mass balance issues. When `basin_wb_aa.txt` looks wrong: climate mapping, curve numbers, missing management, bad soils.
- Performance tips. Build flags, daily versus sub-daily routing, the cost of enabling many outputs.

When in doubt

1. Confirm the executable matches the input format. Use the SWAT+ revision printed on the first line of `simulation.out`.
2. Read `diagnostics.out` end to end. Every line is a missing reference that SWAT+ replaced with a default or skipped.
3. Read `checker.out`. The per-HRU `zmx`, `sumfc`, `sumul`, `tiledrain` columns are the fastest way to spot a soil or option that did not load.

4. Confirm `success.fin` exists. If it does not, the run aborted.

SWAT+ writes runtime messages to three files. The location of the message tells you the type of problem:

- `simulation.out`: console-style status. Successful runs end with `Execution successfully completed`. Failed runs stop mid-stream.
- `diagnostics.out`: missing references and ignored options. Each line is a name that one input file pointed at but that does not exist in the file it was supposed to be in. The run continues, but with a default substituted.
- `checker.out`: resolved per-HRU parameters. Bad values here often mean a soil or option silently failed to load.
- console output to stdout: a small number of `Error:` lines written directly by some readers. These often precede an abort.

Source: error and warning text strings traced to the SWAT+ source at `/drive/c/repositories/swatplus/src/`. Search the source with `grep -rn "write (9001" src/` to see every diagnostics line, and `grep -rn '"Error' src/` to see the stderr lines.

Missing input references (diagnostics.out)

These all share the form `<name> not found in <file>` or `<name> not found (<file>)`. The reader detected that an input pointed at a name that the corresponding database does not contain.

`<name> not found in management.sch`

Source: `landuse_read.f90` line 147.

A row in `landuse.lum` named a management schedule that does not exist in `management.sch`. Fix: open `landuse.lum`, find the row whose `mgt` column holds `<name>`, and either add a matching schedule to `management.sch` or point the row at one that exists.

This is a warning, not a stop. The land use will run without scheduled operations, which means no plant, no fertilize, no harvest. The HRU will not behave as intended.

`<name> not found in plant.ini`

Source: `landuse_read.f90` line 144.

Same pattern, different column. Fix the `plnt_com` column of the offending `landuse.lum` row, or add a row to `plant.ini`.

`<name> not found (landuse.lum)`

Source: `hru_read.f90` line 77.

An HRU in `hru-data.hru` named a `lu_mgt` that does not exist in `landuse.lum`. The HRU runs with no land use, which is rarely what you want.

`<name> not found (soils.sol)`

Source: `hru_read.f90` line 157.

An HRU named a soil that is not in `soils.sol`. The HRU will fall back to a default soil. `checker.out` is the fastest place to confirm: look for `zmx = 2000`, `usle_k = 0`, or any of your soils missing entirely.

`<name> not found (topography.hyd), (hydrograph.hyd), (snow.sno)`

Same family. The pointer in `hru-data.hru` does not match any record in the named file. Verify the name in both files matches exactly. Names are case-sensitive.

<name> not found (initial.cha), (hydrology.cha), (sediment.cha), (nutrients.cha)

Source: `ch_read.f90` lines 97 to 100.

A channel definition references a property record that is missing. The channel will use defaults. If your project has no channels, ignore.

<name> not found (res-init), (res-hyd), (res-release), (res-sed), (res-nut)

Source: `res_read.f90` and `wet_initial.f90`. Same pattern for reservoirs and wetlands. Fix the corresponding pointer.

file not found (wgn), (pgage), (tgage), (sgage), (hgage), (wgage), (petgage), (atmodep)

Source: `cli_staread.f90` lines 72 to 95.

A weather station in `weather-sta.cli` named a gage that does not match any row in the corresponding `.cli` index file. Verify the index file lists the station file under the same name. The Ames_sub1 shipped dataset prints `ames.tem  file not found (tgage)` because the editor wrote the station with a `.tem` extension that the `.tmp` index does not contain.

file not found (basins_carbon.tes)

Source: `carbon_read.f90` line 19.

Optional carbon-test file. Safe to ignore unless you are running the carbon test harness.

WARNING: The variable <name> in the input file carb_coefs.cbn is not a recognized variable.

Source: `carbon_coef_read.f90` line 141.

The reader did not recognize a variable name in `carb_coefs.cbn`. Check spelling against the names in `carbon_coef_read.f90`. The variable is ignored.

Warning: Input file named <name> is missing or null.

Source: `utils.f90` line 69. Emitted by the generic CSV/table reader when a referenced file is `null` or missing. Usually benign because most slots in `file.cio` accept `null`.

Warning: unknown column header named <name>

Source: `utils.f90` line 874. The reader's column-name table did not include the header it found. The column is skipped. Often caused by a file written by a newer or older editor than the executable expects.

Errors written to stdout

Error: The output object <name> in the input file print.prt is not a valid object.

Source: `basin_print_codes_read.f90` line 665.

A row in `print.prt` names an object that is not in the recognized list. Fix the row name. The list of valid objects is in `basin_module.f90`.

Error: <name> print object is duplicated in the input file print.prt. Aborting

Source: `basin_module.f90` line 429.

A `print.prt` row appears twice. Delete the duplicate.

`Error: The number of nmbr_soil_test_layers has not been specified`

Source: `carbon_coef_read.f90` line 120. The `carb_coefs.cbn` file is missing the `nmbr_soil_test_layers` declaration. Add it before the soil-test block.

`Error: The number input soil test layers exceeds nmbr_soil_test_layers of <n>`

Source: `carbon_coef_read.f90` line 127. You declared `nmbr_soil_test_layers = n` and then provided more than `n` soil-test rows. Match them.

Failure modes that are not text errors

Run stops mid-day with no `Execution successfully completed`

Look at the last `Original Simulation mm dd yyyy` line in `simulation.out`. The crash is on or near that date. Common causes:

- Climate file is shorter than the simulation period.
- A divide-by-zero in a hydrology routine (most often caused by zero soil-layer depths or zero plant biomass).
- The compiler-generated `STOP` from a numerical routine (look for an integer code in console output).

`success.fin` is missing

The run did not complete. The model writes `success.fin` only on a clean exit.

Output files are not written

The corresponding row in `print.prt` is set to `n` for that interval. Cross-check against `files_out.out`, which lists every output file the run actually opened.

`Permission denied` (Linux, macOS)

The executable file is not marked executable. Run `chmod +x swatplus-<version>-linux-x86_64`.

`Error opening file ...`

The model is not running from inside the project folder. `cd` into the folder, or pass a full path to a launcher that does so.

Related

- Checker and diagnostics for the file structures.
- Water balance issues for output sanity checks.

When `basin_wb_aa.txt` does not match expectation, the cause is almost always in one of four places: climate input mapping, curve numbers, the management schedule, or soil parameters. Inspect them in that order. Compare the offending values in `checker.out` and `diagnostics.out` against the source of truth in your input files.

What to compare

The basin-average annual water balance from `basin_wb_aa.txt` should approximately satisfy:

```
precip = et + surq_gen + latq + perc + snopack_storage + soil_water_change
```

Where `wateryld = surq_gen + latq + qtile`. Big discrepancies in any one term point at a specific input group.

Symptom: precip is wrong

The reported `precip` does not match the rainfall in the source `.pcp` file.

- Open `weather-sta.cli`. Confirm the station name and that `pgage` points at the right `.cli` index.
- Open the `.cli` index. Confirm it lists the `.pcp` filename that contains the right data.
- Open the `.pcp` file and check the first record. The header carries the `nbyr`, `tstep`, `lat`, `lon`, and `elev` for the station. Lat and lon mismatched with `hru.con` can mean the wrong station was assigned to the HRU.

If `diagnostics.out` contains `<name> file not found (pgage)`, the climate mapping failed entirely and SWAT+ generated rainfall from the weather generator. The reported `precip` will not match any observed dataset.

References: *.cli, *.pcp.

Symptom: surq_gen is too high or too low

Surface runoff is controlled by:

1. The curve number for the (soil hydrologic group, land cover) pair, taken from `cntable.lum` and modified by `cn3_swf` in `hydrology.hyd`.
2. The soil-water content, computed from `soils.sol` field capacity and saturation.
3. The infiltration method: `gampt = 0` (curve number) or `gampt = 1` (Green-Ampt) in `codes.bsn`.

Checks:

- Open `checker.out`. Confirm `hydgrp` (A, B, C, D) per soil matches what your `soils.sol` declares. A misclassification flips runoff by tens of mm.
- Open `cntable.lum`. The CN values are looked up by the `cn2` column of the offending `landuse.lum` row. Cross-check the value against a CN table for that land cover / hydrologic group pair.
- Check `cn3_swf` in `hydrology.hyd`. Defaults to zero; a value near 1 will damp runoff response to wet conditions.
- Confirm `gampt` in `codes.bsn` is what you intended.

References: Checker and diagnostics, codes.bsn, landuse.lum.

Symptom: ET is too high or too low

ET depends on the PET method and on plant cover.

- `codes.bsn` field 3 (`pet`): 0 Priestley-Taylor, 1 Penman-Monteith, 2 Hargreaves, 3 file. Penman-Monteith needs solar radiation, humidity, and wind; if those are missing, the model uses generated values.
- `esco` and `epco` in `hydrology.hyd` (per HRU) control soil evaporation and plant uptake compensation. Defaults in `checker.out` are 0.95 and 1.0; deviations from those change the ET partition.
- Plant cover: if the management schedule is wrong, plants never grow, `eplant` stays low, and the run produces more bare-soil evaporation and runoff.

Open `checker.out` and confirm `esco` and `epco` match what you expect. Open `hru_pw_aa.txt` and inspect biomass and LAI; near-zero biomass on a cropped HRU means management did not fire.

References: codes.bsn, hru.md.

Symptom: management never fires

When `landuse.lum` points at a schedule that does not exist, `diagnostics.out` writes:

```
<lum_name>  <mgt_name>  not found in management.sch
```

The HRU runs with no operations: no plant, no fertilize, no harvest. `hru_pw_aa.txt` will show zero biomass and zero yield. `crop_yld_aa.txt` will be empty or report a residual.

Fix by either matching `landuse.lum%mgt` to a schedule name in `management.sch`, or adding the schedule.

Reference: *.dtl covers decision-table-driven operations.

Symptom: percolation is zero or saturated

Percolation depends on the soil profile in `soils.sol`:

- `sumfc` (column in `checker.out`): the total field capacity over the profile. A near-zero value means the soil-layer field capacities did not load.
- `sumul`: the saturation upper limit. Should be larger than `sumfc`.
- The `perco` coefficient in `hydrology.hyd` scales the percolation rate. Default 0.5; setting to 0 stops percolation.

A `zmx = 2000` row in `checker.out` for every HRU usually means the per-soil `soils.sol` block was not parsed. Look for warnings in `diagnostics.out` referencing the soil name.

References: soils.sol, checker.out.

Symptom: bad latq, qtile, or surq_runon

These depend on:

- `latq_co` in `hydrology.hyd`. The lateral-flow coefficient. Zero turns lateral flow off. Default in the shipped Ames data is 0.01.
- `tiledrain` flag in `hydrology.hyd`. Confirm `checker.out` shows `0` (no tile) or `1` (tile) as intended.
- Tile drainage parameters: see the `tile` block in `hydrology.hyd` and the `tiledrain.str` structural file.

Symptom: ru_aa.txt and basin_wb_aa.txt disagree

The routing-unit output sums HRU contributions through routing. The basin output sums them directly. A large disagreement means routing is doing something unexpected. Check:

- `rout_unit.def`, `rout_unit.ele`, `rout_unit.rtu`. The routing definition and area weights.
- `chandeg.con` and the channel-routing inputs. A misconfigured channel that has no length or no slope produces bizarre routed values.

Workflow

1. Run the model. Confirm `success.fin` exists.
2. Read `diagnostics.out` end to end. Resolve every line.
3. Open `checker.out`. Per HRU, sanity-check `hydgrp`, `zmx`, `sumfc`, `sumul`, `usle_k`, `esco`, `epco`, `perco`, `latq_co`, `tiledrain`.
4. Open `basin_wb_aa.txt`. Compare each term against your expectation. If precip is wrong, fix climate mapping. If ET is wrong, check PET and esco/epco. If surface runoff is wrong, check CN. If percolation is wrong, check soils.

Related

- Checker and diagnostics for the structure of the diagnostic files.
- Common runtime errors for the exact warning strings.
- Water balance outputs for the column definitions.

SWAT+ is a serial Fortran program. There is no MPI, no OpenMP. The run time is set by the simulation length, the number of objects (HRUs, channels, aquifers), the time step, and how much output is written. The build is parallel; the model is not.

Build with `-j`

CMake passes the parallel-jobs flag to the underlying build tool. Use it on the build step:

```
cmake -B build
cmake --build build -j 8
```

`-j 8` runs eight compile jobs at once. Use a value close to your physical core count. The build is CPU-bound; oversubscribing the I/O does not help.

A clean build of SWAT+ on a recent laptop takes a couple of minutes with `-j 8` and around ten minutes with `-j 1`.

Choose an optimized build type

For production runs, build in Release mode (debug symbols and runtime checks off):

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

This is the default in the SWAT+ `CMakeLists.txt`. If you have configured a Debug build, expect runs to be five to ten times slower.

For development, use Debug or RelWithDebInfo so floating-point checks are enabled.

Daily versus sub-daily routing

`time.sim` field `step`:

- `step = 0`: daily. Fastest.
- `step > 0`: sub-daily. Required for Green-Ampt infiltration (`gampt = 1` in `codes.bsn`) and for sub-daily channel routing.

Sub-daily routing multiplies the inner-loop work by the number of sub-daily steps per day. A 46-year daily run that finishes in ten seconds will take several minutes at hourly. Use sub-daily only when the science needs it. Green-Ampt and hourly stage data are the most common reasons.

References: time.sim, codes.bsn.

The cost of enabling many outputs

Every row in `print.prt` that is set to `daily` writes one record per object per day. The cost grows with the simulation length, the number of objects, and the number of rows.

Concretely:

- 46 years of daily output for 12 HRUs and 10 output rows is 12 * 46 * 365 * 10 = around 2 million records. The model produces these in seconds, but parsing them downstream can take longer than the simulation itself.
- The `aa` (average-annual) interval is essentially free. One record per object at end of run.
- `mon` (monthly) is 12 per year. `yr` (yearly) is 1 per year.

If you only need average-annual results, leave everything at `aa` (as the Ames_sub1 default does). Turn `daily` on only for the variables you will plot.

The CSV-output flag (`csvout = y` in `print.prt`) also adds cost: every output is written twice (once as fixed-width, once as CSV). Disable when not needed.

Reference: print.prt.

Object count

Run time scales roughly linearly with the number of HRUs, channels, aquifers, and routing units in the project. A 12-HRU project (Ames_sub1) finishes in a few seconds. A 5000-HRU regional project takes minutes per simulated decade.

If you are iterating on inputs, prototype on a small subset and scale up only when the inputs are stable.

I/O

SWAT+ reads inputs once at the start and writes outputs progressively during the run. Putting the project folder on a local SSD instead of a network share matters more than CPU when many outputs are enabled.

Profiling

Compile with `-pg` (gfortran) or the Intel compiler's profile flags, run a representative scenario, and inspect with `gprof` or the Intel tools. Most time in a typical project goes to `pl_grow` (plant growth), the soil-water routines, and the channel routing. Adding more daily output rows shifts the balance toward the I/O routines.

Summary

Lever	Effect
`cmake --build build -j N`	Build time, no run-time effect.
`CMAKE_BUILD_TYPE=Release`	Five to ten times faster runs.
`step = 0` in `time.sim`	Daily, fastest.
`step > 0` in `time.sim`	Sub-daily, multiplies inner-loop cost.
Many `daily` rows in `print.prt`	More disk and downstream parsing cost.
Use `aa` only	Cheapest output mode.
Local SSD vs network share	Real difference when many outputs are on.